

复习课讲义

数据科学与大数据技术专业

第 10 讲

1 复习主线

本讲是 W1-W9 的复习课讲义, 把已经学过的概念重新放回同一条主线中. 需要把握的是: 深度学习看起来模型很多, 但最基本的问题一直是如何从数据中学习一个预测函数, 并让这个函数在未见过的数据上也表现稳定.

因此, 本讲的组织顺序仍然从一般机器学习框架出发, 先复习线性模型, 再复习前馈神经网络和反向传播, 然后看 CNN、RNN、GPT/LLM 和 Transformer 只是分别围绕不同数据结构、生成方式或信息汇聚方式改造了网络系统.

1. **机器学习基本框架**: 数据、模型、学习准则、优化算法, 以及训练集、验证集和测试集的分工.
2. **线性模型**: 线性回归、Logistic 回归、Softmax 回归、感知器和支持向量机 (SVM) 都有着线性的决策平面.
3. **神经网络**: 前馈网络按层计算 $\mathbf{z}^{(l)}$ 和 $\mathbf{a}^{(l)}$, 反向传播用链式法则计算梯度.
4. **CNN 与 RNN**: CNN 用局部连接和权重共享处理空间结构, RNN 用隐藏状态 $\mathbf{h}_t$ 处理序列历史.
5. **训练技巧、LLM 与注意力**: 优化器、初始化、归一化和正则化帮助训练更稳定; GPT/LLM 用下一个 token 预测生成文本; 注意力用 QKV 形式动态汇聚相关信息.

如果细节都忘了: 最应该记住的是, 本课程反复出现的是同一个学习闭环:

数据 $\rightarrow$ 模型 $\rightarrow$ 损失 $\rightarrow$ 梯度优化 $\rightarrow$ 验证泛化.

后面的神经网络、卷积网络、循环网络和注意力模型, 都是在改变模型结构或训练方式, 不是脱离这个闭环的新问题. 理解一个模型时, 可以先判断它属于闭环中的哪一环: 它是在改变函数集合, 改变损失函数, 还是改变训练和泛化控制的方法.

问题思考:

- 给定一个新算法描述, 如何判断其中的模型、损失函数和优化算法分别是什么?
- 为什么训练误差很低不能直接说明模型在真实数据分布上表现好?

2 机器学习与线性模型

这一节复习 W1-W3 的内容. 可以把线性模型当作所有后续模型的最小版本: 输入特征 $\mathbf{x}$ 先被映射成一个得分, 损失函数再根据得分和标签之间的差异给出梯度. 后面神经网络只是把这个得分函数从一层线性变换扩展成多层复合函数.

1. 四个基本要素

- 数据通常记为样本对 $(\mathbf{x}^{(n)}, y^{(n)})$. 在监督学习中, $\mathbf{x}^{(n)}$ 是输入特征, $y^{(n)}$ 是标签, 训练集给出的只是有限样本, 不是完整的数据分布.
- 模型是函数集合 $\mathcal{F}$ 中的一个函数. 选择线性模型、神经网络、CNN、RNN 或 Transformer, 本质上是在选择不同的函数集合, 也就是规定模型能表达什么样的输入输出关系.
- 学习准则用损失函数 $\mathcal{L}$ 衡量预测与真实标签的差异. 回归问题常用平方损失, 分类问题常用交叉熵或间隔损失.
- 优化算法根据损失对参数的梯度更新参数, 最基本形式为

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t).$$

这里 η 是学习率. 若 η 太小, 下降很慢; 若 η 太大, 参数可能在最优点附近震荡甚至发散.

2. 经验风险与泛化

- 真正希望最小化的是期望风险

$$\mathcal{R}(f) = \mathbb{E}_{(\mathbf{x}, y) \sim P_{\text{data}}} [\mathcal{L}(y, f(\mathbf{x}))],$$

但真实分布未知, 实际训练只能最小化训练集上的经验风险

$$\hat{\mathcal{R}}_D(f) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(y^{(n)}, f(\mathbf{x}^{(n)})).$$

经验风险是期望风险的样本平均近似. 样本越少、模型越复杂, 这种近似就越容易被模型利用. 过拟合指模型在训练集上损失很低, 但在未见过的数据上表现明显变差: 它不只是学到了稳定规律, 还把训练样本中的噪声、偶然相关和重复细节也当作规律记了下来.

- 训练集用于学习参数, 验证集用于模型选择和调参, 测试集只用于最终评估. 若只看训练集损失, 很容易把过拟合误认为模型能力强. 一个常见现象是 training loss 持续下降, 但 validation loss 先下降后上升, 这通常说明模型开始记忆训练集中的偶然细节.

3. 评价指标与验证协议

- 在二分类中, 混淆矩阵把样本分为真正例 TP 、假正例 FP 、真负例 TN 和假负例 FN . 三个常用指标为

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}, \quad \text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}.$$

其中 Accuracy 通常译为准确率, 衡量总体预测正确的比例; Precision 才是精确率, 回答“模型报出的正例中有多少是真的”; Recall 是召回率, 回答“真正例中有多少被找回”.

- Precision 和 Recall 的侧重点不同。例如真实正例有 100 个，模型只报出其中 50 个且没有误报，则 $TP = 50, FP = 0, FN = 50$ ，Precision 为 100%，但 Recall 只有 50%；这说明“报出的都对”不等于“该找的都找到了”。反过来，如果模型把所有样本都标为正例，则 $FN = 0$ ，Recall 为 100%，但 FP 可能很大，Precision 会很低。
- 在类别比较均衡、错误代价接近时，Accuracy 是直观指标；但在异常检测、疾病筛查等正类很少的任务中，总体 Accuracy 可能严重误导。例如异常样本只占 2% 时，总是预测“正常”也能得到 98% Accuracy，但异常召回率为 0。
- 对类别不平衡任务，至少要同时查看混淆矩阵、Recall、Precision、 F_1 或 PR-AUC 指标。 F_1 是 Precision 和 Recall 的调和平均，PR-AUC 比 ROC-AUC 更能反映稀有正类场景下的正类识别质量。
- 训练集、验证集、测试集的分工还涉及“选择偏差”。验证集可以反复用于选择网络深度、学习率、正则化强度和早停轮次，但如果最后仍把同一个验证集分数当作最终性能，就等于让模型和调参过程共同适配了验证集。因此最终报告应使用独立测试集，测试集不参与梯度更新，也不参与多轮调参决策。
- 在时间序列或设备数据中，划分方式也会影响结论。若用滑动窗口构造样本，高度重叠的相邻窗口不应随机散落到训练集和测试集中；否则测试集可能包含与训练窗口几乎相同的信息，使泛化表现被高估。

4. 线性回归

- 线性回归使用

$$\hat{y} = f(\mathbf{x}; \mathbf{w}) = \mathbf{w}^\top \mathbf{x},$$

常配合平方损失。若加入 ℓ_2 正则化，目标可写为

$$\frac{1}{2} \|\mathbf{y} - \mathbf{X}^\top \mathbf{w}\|^2 + \frac{\lambda}{2} \|\mathbf{w}\|^2.$$

这里第一项要求预测值贴近真实值，第二项惩罚过大的参数。当两个特征高度相关时，没有正则化的解可能用一正一负的很大权重互相抵消； ℓ_2 正则化会倾向于选择更平滑、更小范数的权重。

- 从概率视角看，高斯噪声假设对应平方损失；高斯先验对应 ℓ_2 正则化。这个视角的作用是说明：损失函数和正则化项并不是随意添加的，它们可以对应数据噪声和参数规模的建模假设。

5. 分类模型

- 线性分类器先计算得分 $f_c(\mathbf{x}) = \mathbf{w}_c^\top \mathbf{x} + b_c$ ，再根据得分给出类别、概率或间隔。需要区分得分和概率：得分可以是任意实数，概率必须非负且和为 1。
- Logistic 回归用于二分类：

$$\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x} + b), \quad \sigma(z) = \frac{1}{1 + \exp(-z)}.$$

这里的 $\sigma(\cdot)$ 是 Sigmoid 函数。它把一个标量线性得分压缩到 $(0, 1)$ ，因而可以解释为类别 1 的概率。

- Softmax 回归用于多分类:

$$\hat{y}_c = \frac{\exp(z_c)}{\sum_{j=1}^C \exp(z_j)}.$$

这里的归一化指数映射是 Softmax 函数. 它同时作用在所有类别得分 $\mathbf{z} = (z_1, \dots, z_C)$ 上, 输出非负且和为 1 的概率分布. 与交叉熵结合时, 得分层梯度有简洁形式 $\hat{y}_c - y_c$.

- 感知器只在样本分错时更新; SVM 通过最大间隔寻找更稳定的分类边界. 二者都不直接输出概率.

6. 例子: 线性分类器的目标与更新

- Logistic 回归可以从二分类似然得到交叉熵. 对样本 $(\mathbf{x}^{(i)}, y^{(i)})$, 若 $\hat{y}^{(i)} = \sigma(\mathbf{w}^\top \mathbf{x}^{(i)} + b)$, 则二分类交叉熵为

$$J(\mathbf{w}, b) = -\frac{1}{N} \sum_{i=1}^N [y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log (1 - \hat{y}^{(i)})].$$

对一个样本, 梯度可写成

$$\frac{\partial L}{\partial \mathbf{w}} = (\hat{y} - y)\mathbf{x}, \quad \frac{\partial L}{\partial b} = \hat{y} - y.$$

直觉上, 若 $y = 1$, 则 $\mathbf{x}$ 是正例. 如果当前参数给出的 $\hat{y} < 1$, 模型对正类的概率估计偏低, 梯度下降就应该把线性得分 $z = \mathbf{w}^\top \mathbf{x} + b$ 往上推. 数学上也正是这样: 当 $y = 1$ 时,

$$\mathbf{w}_{\text{new}} = \mathbf{w} + \eta(1 - \hat{y})\mathbf{x}, \quad b_{\text{new}} = b + \eta(1 - \hat{y}),$$

因而

$$z_{\text{new}} = \mathbf{w}_{\text{new}}^\top \mathbf{x} + b_{\text{new}} = z + \eta(1 - \hat{y})(\|\mathbf{x}\|^2 + 1) > z.$$

由于 Sigmoid 是单调递增函数, z 增大就会使 $\hat{y} = \sigma(z)$ 增大. 例如

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \quad b = 0,$$

此时 $z = -1$, $\hat{y} = \sigma(-1) \approx 0.269$. 对正例来说这个概率偏低, 所以一步梯度下降会提高 z , 从而提高正类概率.

- 在深度学习中, logits 通常指进入 Sigmoid 或 Softmax 之前的原始得分. 它们还不是概率: 可以为负, 不要求和为 1, 只通过相对大小影响分类结果. Softmax 回归先把 logits 转成概率, 再写出多分类交叉熵:

$$J(\mathbf{W}, \mathbf{b}) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_k^{(i)} \log \hat{y}_k^{(i)}.$$

如果 logits 为 $(2, 1, 0)$, 且真实类别是第 1 类, 则

$$\hat{\mathbf{y}} = \frac{1}{\exp(2) + \exp(1) + 1} \begin{bmatrix} \exp(2) \\ \exp(1) \\ 1 \end{bmatrix}, \quad \frac{\partial L}{\partial \mathbf{z}} = \hat{\mathbf{y}} - \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}.$$

这个向量就是损失对 logits 的梯度. 代入数值可得

$$\hat{\mathbf{y}} \approx \begin{bmatrix} 0.665 \\ 0.245 \\ 0.090 \end{bmatrix}, \quad \frac{\partial L}{\partial \mathbf{z}} \approx \begin{bmatrix} -0.335 \\ 0.245 \\ 0.090 \end{bmatrix}.$$

梯度下降按 $\mathbf{z} \leftarrow \mathbf{z} - \eta \frac{\partial L}{\partial \mathbf{z}}$ 的方向改变 logits, 因而第 1 类 logit 会增加, 第 2、3 类 logit 会降低. 由于第 2 类当前概率大于第 3 类, 它被压低的幅度也更大. 这说明交叉熵梯度的作用不是简单地“把正确类调大”, 而是在所有类别之间重新分配相对得分, 使真实类别的 Softmax 概率上升.

- 感知器的更新只由错分样本触发. 若 $y \in \{-1, +1\}$ 且 $y\mathbf{w}_t^\top \mathbf{x} \leq 0$, 则

$$\mathbf{w}_{t+1} = \mathbf{w}_t + y\mathbf{x}, \quad y\mathbf{w}_{t+1}^\top \mathbf{x} = y\mathbf{w}_t^\top \mathbf{x} + \|\mathbf{x}\|^2.$$

因而更新会把该样本朝正确一侧推.

- SVM 关注间隔和支持向量. 线性可分情形下要求

$$y^{(n)} (\mathbf{w}^\top \mathbf{x}^{(n)} + b) \geq 1.$$

满足等号的样本是支持向量, 它们决定最大间隔分类边界. 更完整地说, 样本到分类超平面的几何间隔为

$$\gamma^{(n)} = \frac{y^{(n)} (\mathbf{w}^\top \mathbf{x}^{(n)} + b)}{\|\mathbf{w}\|}.$$

如果只把 $\mathbf{w}, b$ 同时乘以同一个正数, 分子和分母会同步放大, 几何间隔并不会真正变大. SVM 固定函数间隔后再最小化 $\|\mathbf{w}\|$, 等价于最大化最近样本的几何间隔, 因而通常对小扰动更鲁棒.

问题思考:

- Logistic 回归和 SVM 都能做二分类, 它们的输出解释和学习目标有什么不同?
- 为什么验证集不能像训练集一样反复用于更新参数?

3 神经网络与反向传播

这一节复习 W4 的主线. 线性模型只有一次线性变换, 前馈神经网络把多次“线性变换 + 非线性激活”复合起来. 反向传播要解决的问题是: 当损失 L 已经由最后一层输出给出时, 怎样系统地把梯度传回每一层参数.

1. 前馈神经网络

- 前馈网络按层计算

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = f_l(\mathbf{z}^{(l)}).$$

其中 $\mathbf{a}^{(0)} = \mathbf{x}$, $\mathbf{W}^{(l)}$ 把上一层激活映射到当前层净输入, $\mathbf{b}^{(l)}$ 是偏置.

- 若所有层都是线性变换, 多层网络仍等价于一层线性变换. 因此, 非线性激活函数是神经网络表达复杂函数的关键. 可以用两层线性层说明:

$$\mathbf{W}^{(2)} \left(\mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)} \right) + \mathbf{b}^{(2)} = \left(\mathbf{W}^{(2)} \mathbf{W}^{(1)} \right) \mathbf{x} + \left(\mathbf{W}^{(2)} \mathbf{b}^{(1)} + \mathbf{b}^{(2)} \right).$$

没有非线性时, “深”并不会带来本质上新函数形式.

2. 常见激活函数

- Sigmoid 输出在 $(0, 1)$, 适合把标量得分解释成二分类概率, 但在输入绝对值很大时容易饱和, 导数接近 0.
- Tanh 输出在 $(-1, 1)$, 比 Sigmoid 更零中心, 但饱和区仍会导致梯度变小.
- 如果初始权重尺度过大, 前向传播中的净输入 $z = \mathbf{w}^\top \mathbf{x} + b$ 绝对值可能很大, Sigmoid 或 Tanh 会进入饱和区. 此时 $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ 或 $\tanh'(z) = 1 - \tanh^2(z)$ 接近 0, 反向传播时梯度被压小, 深层网络会更难训练.
- ReLU 计算简单, 正半轴导数为 1, 常用于深层网络; 负半轴导数为 0, 如果一个神经元长期落在负半轴, 就可能出现死亡 ReLU 问题.
- 需要注意, ReLU 的输出满足 $0 \leq a < \infty$, 正半轴没有上界; “输出限制在 $(0, 1)$ ”是 Sigmoid 的性质, 不是 ReLU 的优势. ReLU 的常见优势主要是形式简单、正半轴梯度稳定, 以及负半轴可产生稀疏激活.

3. 反向传播

- 定义误差项

$$\delta^{(l)} = \frac{\partial L}{\partial \mathbf{z}^{(l)}}.$$

它表示损失对第 l 层净输入的敏感程度. 若 $\mathbf{z}^{(l)} \in \mathbb{R}^{d_l}$, 则 $\delta^{(l)} \in \mathbb{R}^{d_l}$.

- 则第 l 层参数梯度为

$$\frac{\partial L}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} (\mathbf{a}^{(l-1)})^\top, \quad \frac{\partial L}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}.$$

这个公式的维度很重要: 若 $\delta^{(l)}$ 是 $d_l \times 1$, $\mathbf{a}^{(l-1)}$ 是 $d_{l-1} \times 1$, 则外积给出 $d_l \times d_{l-1}$ 的矩阵, 正好与 $\mathbf{W}^{(l)}$ 同形状.

- 误差项从后一层递推到前一层:

$$\delta^{(l)} = f'_l(\mathbf{z}^{(l)}) \odot \left(\left(\mathbf{W}^{(l+1)} \right)^\top \delta^{(l+1)} \right).$$

- 反向传播不是新的求导规则, 而是把链式法则组织成可复用、可高效计算的算法. 前向传播保存中间量, 反向传播复用这些中间量逐层计算梯度.

4. 标量 ReLU 网络的手算模板

- 对小型网络手算题, 可以先把它当作普通函数求导题. 例如 $x = 1$, 两个隐藏单元为

$$a_1 = \text{ReLU}(w_1), \quad a_2 = \text{ReLU}(w_2),$$

输出为

$$\hat{y} = 2a_1 - a_2 + \frac{1}{2}, \quad L = \frac{1}{2}(\hat{y} - 1)^2.$$

直接把损失写成参数的函数:

$$L(w_1, w_2) = \frac{1}{2} \left(2 \operatorname{ReLU}(w_1) - \operatorname{ReLU}(w_2) - \frac{1}{2} \right)^2.$$

这类题只需先判断 ReLU 的分段. 在当前点 $(w_1, w_2) = (1, 1)$, 两个 ReLU 都处在正半轴, 因而局部上

$$L(w_1, w_2) = \frac{1}{2} \left(2w_1 - w_2 - \frac{1}{2} \right)^2.$$

令 $g(w_1, w_2) = 2w_1 - w_2 - \frac{1}{2}$, 则 $g(1, 1) = \frac{1}{2}$, 梯度为

$$\nabla L(1, 1) = g(1, 1) \begin{bmatrix} 2 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 \\ -\frac{1}{2} \end{bmatrix}.$$

若学习率为 0.1, 一步梯度下降为

$$(w_1, w_2) \leftarrow (1, 1) - 0.1 \left(1, -\frac{1}{2} \right) = (0.9, 1.05).$$

因为平方损失非负, 所有满足 $\hat{y} = 1$ 的点都是全局极小值; 在当前正半轴分段内, 直线 $w_2 = 2w_1 - \frac{1}{2}$ 上的点都达到 $L = 0$, 所以全局极小值通常不唯一.

- 带偏置的单隐藏单元也按同一模板计算. 设

$$z = wx + b, \quad a = \operatorname{ReLU}(z), \quad \hat{y} = va + c, \quad L = \frac{1}{2}(\hat{y} - y)^2.$$

若 $z > 0$, 则 $\operatorname{ReLU}'(z) = 1$, 梯度为

$$\frac{\partial L}{\partial v} = ra, \quad \frac{\partial L}{\partial c} = r, \quad \frac{\partial L}{\partial w} = rvx, \quad \frac{\partial L}{\partial b} = rv.$$

例如 $x = 1, y = 2, w = 1, b = 0, v = 2, c = -1$ 时, $z = 1, a = 1, \hat{y} = 1, r = -1$, 所以梯度为 $(-2, -2, -1, -1)$, 分别对应 w, b, v, c ; 学习率 0.1 下更新为 $w = 1.2, b = 0.2, v = 2.1, c = -0.9$. 若某个样本满足 $z < 0$, 则 $a = 0$ 且 $\operatorname{ReLU}'(z) = 0$, 该样本对 w, b 的梯度为 0, 这正是 ReLU 负半轴零梯度和死亡 ReLU 风险的来源.

5. 例子: 两层全连接网络手算梯度

- 下面例子把梯度计算放到一个两层全连接网络里. 这里“两层”指有两层带参数的线性变换: 一个隐藏层和一个输出层, 每层都有两个神经元:

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \mathbf{W}^{(1)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} 0 \\ -1 \end{bmatrix},$$

$$\mathbf{W}^{(2)} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad \mathbf{b}^{(2)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

隐藏层用 ReLU, 输出层线性, 损失为

$$L = \frac{1}{2} \|\hat{\mathbf{y}} - \mathbf{y}\|^2.$$

- 前向传播:

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \mathbf{a}^{(1)} = \text{ReLU}(\mathbf{z}^{(1)}) = \begin{bmatrix} 1 \\ 1 \end{bmatrix},$$

$$\hat{\mathbf{y}} = \mathbf{W}^{(2)} \mathbf{a}^{(1)} + \mathbf{b}^{(2)} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}, \quad L = 1.$$

因为两个隐藏单元的净输入都为正, 所以这里 ReLU 导数的两个分量都为 1.

- 输出层误差项和梯度:

$$\delta^{(2)} = \hat{\mathbf{y}} - \mathbf{y} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \quad \frac{\partial L}{\partial \mathbf{W}^{(2)}} = \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix}, \quad \frac{\partial L}{\partial \mathbf{b}^{(2)}} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

- 第一层误差项和梯度:

$$\delta^{(1)} = \left(\mathbf{W}^{(2)} \right)^\top \delta^{(2)} \odot \text{ReLU}'(\mathbf{z}^{(1)}) = \begin{bmatrix} 0 \\ 2 \end{bmatrix},$$

$$\frac{\partial L}{\partial \mathbf{W}^{(1)}} = \delta^{(1)} \mathbf{x}^\top = \begin{bmatrix} 0 & 0 \\ 2 & 4 \end{bmatrix}, \quad \frac{\partial L}{\partial \mathbf{b}^{(1)}} = \begin{bmatrix} 0 \\ 2 \end{bmatrix}.$$

这一步体现了链式法则: 输出层误差先乘以 $\left(\mathbf{W}^{(2)} \right)^\top$ 回传到隐藏层, 再乘以 ReLU 的导数.

- 若学习率 $\eta = 0.1$, 则第一步梯度下降会给出

$$\mathbf{W}^{(2)} \leftarrow \mathbf{W}^{(2)} - 0.1 \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix}, \quad \mathbf{b}^{(2)} \leftarrow \mathbf{b}^{(2)} - 0.1 \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

记住这类问题的核心: 每层权重梯度都是当前层误差项乘以上一层激活的转置.

问题思考:

- 为什么没有激活函数时, 多层全连接网络仍然不能表达非线性决策边界?
- 在两层网络例子中, 若某个隐藏单元的 $z^{(1)} < 0$, 它对第一层梯度会产生什么影响?

4 卷积神经网络

这一节复习 W5 的内容. CNN 的核心是把图像中的空间结构作为先验放进网络: 相邻像素更相关, 同一种局部模式可以出现在不同位置, 空间尺寸可以逐步压缩.

先约定术语: 本节后面提到的“卷积”, 都采用深度学习框架中的实际计算方式, 也就是数学上通常称为互相关的运算: 卷积核不进行翻转, 而是按原方向在输入窗口上逐元素相乘并求和. 下面不再区分这两个名称, 统一称为卷积.

1. 为什么使用 CNN

- 图像等数据具有局部结构. 若直接展平成向量并接全连接层, 参数量大, 且空间邻接关系容易被破坏. 例如一张 $1000 \times 1000 \times 3$ 的图像接到 1000 个隐藏单元, 全连接层权重数约为

$$1000 \times 1000 \times 3 \times 1000 = 3 \times 10^9,$$

这还没有计算偏置, 参数量已经很难训练.

- CNN 的核心是局部连接、权重共享和下采样. 局部连接表示一个输出只看输入中的一个小邻域; 权重共享表示同一个卷积核在不同位置重复使用; 下采样表示通过池化或步长 (stride) 大于 1 的卷积降低空间尺寸.
- 这三个思想对应三个好处: 局部连接保留邻域结构, 权重共享显著降低参数量并利用模式可平移出现的先验, 下采样让后续层在更小的特征图上计算.

2. 卷积与输出尺寸

- 按上述约定, 对二维输入 $\mathbf{X} \in \mathbb{R}^{H \times W}$, 设边界填充 (padding) 后的输入为 $\tilde{\mathbf{X}}$, 卷积核大小 (kernel size) 为 $U \times V$, 卷积核为 $\mathbf{K} \in \mathbb{R}^{U \times V}$, 偏置为 b , 步长为 S_h, S_w , 则单通道卷积输出可写为

$$Y_{i,j} = b + \sum_{u=0}^{U-1} \sum_{v=0}^{V-1} K_{u,v} \tilde{X}_{iS_h+u, jS_w+v}.$$

多通道输入时, 每个输出通道会对所有输入通道分别做这种局部乘加, 再把结果相加.

- 一个手算例子可以帮助理解窗口如何移动. 设

$$\mathbf{X} = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 1 & 1 \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} 1 & 0 \\ -1 & 2 \end{bmatrix}, \quad b = 0,$$

步长为 1, 填充为 0. 左上角输出来自左上 2×2 窗口:

$$Y_{0,0} = 1 \cdot 1 + 0 \cdot 2 + (-1) \cdot 0 + 2 \cdot 1 = 3.$$

其余三个位置同理:

$$Y_{0,1} = 1 \cdot 2 + 0 \cdot 0 + (-1) \cdot 1 + 2 \cdot 3 = 7, \quad Y_{1,0} = 1 \cdot 0 + 0 \cdot 1 + (-1) \cdot 2 + 2 \cdot 1 = 0,$$

$$Y_{1,1} = 1 \cdot 1 + 0 \cdot 3 + (-1) \cdot 1 + 2 \cdot 1 = 2.$$

因而输出为

$$\mathbf{Y} = \begin{bmatrix} 3 & 7 \\ 0 & 2 \end{bmatrix}.$$

- 对二维输入 $\mathbf{X} \in \mathbb{R}^{H \times W}$, 卷积核大小为 $U \times V$, 填充为 P_h, P_w , 步长为 S_h, S_w , 输出尺寸为

$$H_{\text{out}} = \left\lfloor \frac{H - U + 2P_h}{S_h} \right\rfloor + 1, \quad W_{\text{out}} = \left\lfloor \frac{W - V + 2P_w}{S_w} \right\rfloor + 1.$$

公式中的分子表示卷积核能移动的有效范围, 除以步长表示移动次数, 最后的 +1 是初始位置本身.

- 这里容易混淆三件事: 卷积核大小决定单层每个输出看到的局部窗口大小; 步长决定窗口每次移动多远; 填充决定边界外补多少圈. 填充可以保持空间尺寸, 步长通常会压缩空间尺寸.
- 池化也有类似的窗口移动公式. 对每个通道单独做池化, 若输入空间尺寸为 $H \times W$, 池化窗口大小为 $R_h \times R_w$, 填充为 $P_h^{\text{pool}}, P_w^{\text{pool}}$, 步长为 $S_h^{\text{pool}}, S_w^{\text{pool}}$, 则输出空间尺寸为

$$H_{\text{pool}} = \left\lfloor \frac{H - R_h + 2P_h^{\text{pool}}}{S_h^{\text{pool}}} \right\rfloor + 1, \quad W_{\text{pool}} = \left\lfloor \frac{W - R_w + 2P_w^{\text{pool}}}{S_w^{\text{pool}}} \right\rfloor + 1.$$

以最大池化为例, 若输入第 c 个通道为 $\mathbf{X}^{(c)}$, 则

$$Y_{i,j,c} = \max_{0 \leq u < R_h, 0 \leq v < R_w} X_{iS_h^{\text{pool}}+u, jS_w^{\text{pool}}+v}^{(c)}.$$

平均池化只是把上式中的最大值换成窗口内平均值. 池化通常只改变空间尺寸, 不会自动改变通道数.

3. 结构理解

- 卷积层主要生成局部特征, 池化层或步长卷积主要压缩空间尺寸. 多个输出通道可以理解为多个不同的局部特征检测器.
- 多层堆叠会扩大感受野, 使特征从边缘、纹理逐步组合为更高层语义. 注意卷积核大小和感受野不是同一个概念: 单层卷积核可能只有 3×3 , 但经过多层堆叠后, 高层特征已经间接依赖更大的输入区域.
- LeNet、AlexNet、Inception 和 ResNet 都是 CNN 结构发展的例子. 其中 ResNet 的残差块可写为

$$\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}.$$

其反向传播可写为

$$\frac{\partial L}{\partial \mathbf{x}} = \frac{\partial L}{\partial \mathbf{y}} \left(\frac{\partial \mathcal{F}(\mathbf{x})}{\partial \mathbf{x}} + \mathbf{I} \right),$$

其中 $\mathbf{I}$ 提供了直接梯度路径.

4. 例子: CNN 输出尺寸与参数量

- 卷积层参数量为

$$k_H k_W C_{\text{in}} C_{\text{out}} + C_{\text{out}}.$$

其中最后一项是每个输出通道一个偏置.

- 卷积、ReLU 和池化可以连在一起手算. 设输入为 5×5 单通道图像, 卷积核为

$$\mathbf{K} = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}, \quad b = 0,$$

步长为 1, 不填充, 窗口与卷积核逐元素相乘后直接求和. 因为

$$H_{\text{out}} = W_{\text{out}} = \left\lfloor \frac{5-3}{1} \right\rfloor + 1 = 3,$$

完整卷积特征图为 3×3 . 若只看左上 2×2 个卷积位置, 可以把对应局部卷积结果写成矩阵

$$\mathbf{Z}_{\text{tl}} = \begin{bmatrix} 4 & 5 \\ 4 & 3 \end{bmatrix}.$$

这些值都为正, 经过 ReLU 后矩阵不变:

$$\text{ReLU}(\mathbf{Z}_{\text{tl}}) = \begin{bmatrix} 4 & 5 \\ 4 & 3 \end{bmatrix}.$$

再做 2×2 最大池化、步长 2, 左上输出为

$$\max \begin{bmatrix} 4 & 5 \\ 4 & 3 \end{bmatrix} = 5.$$

该卷积核左列为负、右列为正, 本质上近似检测左右明暗变化; 右侧更亮、左侧更暗时响应为正, 因而可理解为检测竖直边缘的一类局部特征.

- 池化层通常没有可学习参数, 但会改变空间尺寸. 对 $31 \times 31 \times 8$ 特征图做 2×2 最大池化, 步长 2, 不填充且不使用不完整窗口时,

$$H_{\text{out}} = W_{\text{out}} = \left\lfloor \frac{31-2}{2} \right\rfloor + 1 = 15,$$

输出为 $15 \times 15 \times 8$. 注意池化不会自动把通道数翻倍, 通道数仍由输入特征图或后续卷积层决定.

- 一个小型分类 CNN 的参数量也可以逐层核对. 输入 $32 \times 32 \times 3$, 卷积核 3×3 , 输出通道 8, 步长 1, 填充 1 时, 卷积输出为 $32 \times 32 \times 8$, 参数量为

$$3 \times 3 \times 3 \times 8 + 8 = 224.$$

之后接 2×2 最大池化、步长 2, 得到 $16 \times 16 \times 8$, Flatten 长度为 $16 \times 16 \times 8 = 2048$. 若最后接 10 类全连接层, 参数量为

$$2048 \times 10 + 10 = 20490.$$

这个例子再次说明: 卷积层通过局部连接和权重共享控制参数量, 但 Flatten 后的全连接层参数量可能迅速增大.

- 简单例子: 输入为 $128 \times 128 \times 3$, 使用 16 个 3×3 卷积核, 步长 1, 填充 0. 参数量为

$$3 \times 3 \times 3 \times 16 + 16 = 448.$$

输出空间尺寸为

$$\frac{128 - 3 + 2 \times 0}{1} + 1 = 126,$$

所以输出为 $126 \times 126 \times 16$.

- 组合结构可以把输出尺寸和参数量连着算. 若输入为 $128 \times 128 \times 3$:
 - 卷积 3×3 , 16 通道, 步长 1, 填充 1: 输出 $128 \times 128 \times 16$, 参数 448.
 - 最大池化 2×2 , 步长 2: 输出 $64 \times 64 \times 16$.
 - 卷积 3×3 , 32 通道, 步长 1, 填充 0: 输出 $62 \times 62 \times 32$, 参数 $3 \times 3 \times 16 \times 32 + 32 = 4640$.
 - 最大池化 2×2 , 步长 2: 输出 $31 \times 31 \times 32$.
 - 接 10 类全连接层: 参数 $31 \times 31 \times 32 \times 10 + 10 = 307530$.

总参数量为 $448 + 4640 + 307530 = 312618$. 这个例子也说明, 前面卷积层参数量不大, 后面如果直接接全连接层, 参数量仍可能突然增大.

问题思考:

- 为什么卷积层的参数量与输入图像的 H, W 无直接乘法关系, 但全连接层通常有?
- 填充、步长和池化分别会怎样影响特征图的空间尺寸?

5 循环神经网络

这一节复习 W6 的内容. RNN 处理的是序列数据, 关键是当前输出不能只看当前输入, 还要利用之前时间步留下来的状态. 因此, RNN 的隐藏状态可以看作对历史信息的压缩表示.

1. RNN 的基本形式

- 序列模型需要利用历史信息. RNN 用隐藏状态 $\mathbf{h}_t$ 压缩过去输入:

$$\mathbf{z}_t = \mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t + \mathbf{b}, \quad \mathbf{h}_t = f(\mathbf{z}_t),$$

$$\hat{\mathbf{y}}_t = g(\mathbf{V}\mathbf{h}_t + \mathbf{c}).$$

- 与前馈网络相比, 关键新增项是 $\mathbf{U}\mathbf{h}_{t-1}$. 它让同一个当前输入在不同历史下产生不同状态.
- 一个最小例子可以只看标量情形:

$$h_t = \tanh(0.5h_{t-1} + x_t), \quad h_0 = 0.$$

若序列为 $x_1 = 1, x_2 = 0$, 则

$$h_1 = \tanh(1), \quad h_2 = \tanh(0.5 \tanh(1)).$$

若序列为 $x_1 = 0, x_2 = 0$, 则 $h_1 = h_2 = 0$. 两个序列在第二步的当前输入同为 0, 但隐藏状态不同, 这正是历史信息起作用的地方.

2. BPTT 与梯度问题

- RNN 沿时间展开后, 每个时间步使用同一组参数 $\mathbf{U}, \mathbf{W}, \mathbf{V}$. 因此, 共享参数的梯度需要累加所有时间步的贡献. 这就是随时间反向传播 (BPTT).

- 长程依赖困难来自反向传播中的矩阵和激活导数连乘. 对简单 RNN, 一步时间传播的局部 Jacobian 包含 $\text{diag}(f'(z_t))U$. 若多数因子尺度小于 1, 梯度消失; 若多数因子尺度大于 1, 梯度爆炸.
- 梯度截断主要处理梯度爆炸造成的数值不稳定, 不能直接恢复已经消失的长程梯度.

3. 问题: RNN 梯度为什么会消失或爆炸

- 设简单 RNN 为

$$z_t = U h_{t-1} + W x_t + b, \quad h_t = f(z_t).$$

若只看时刻 T 的损失对较早隐藏状态的影响, 链式法则会出现

$$\frac{\partial L_T}{\partial h_t} = \frac{\partial L_T}{\partial h_T} \prod_{k=t+1}^T \frac{\partial h_k}{\partial h_{k-1}}.$$

- 如果这些 Jacobian 因子的范数多数小于 1, 梯度会随时间距离指数衰减; 如果多数大于 1, 梯度会指数放大. 这不是某一个时间步的错误, 而是长时间链式相乘带来的整体效应.

4. LSTM 与 GRU

- LSTM 引入细胞状态 c_t , 用输入门、遗忘门和输出门控制写入、保留和输出. 直观上, 细胞状态提供了一条更容易保留长期信息的路径.
- GRU 使用更新门和重置门控制隐藏状态 h_t , 结构比 LSTM 更简单.
- 二者的共同思想都是门控: 模型学习什么该保留, 什么该丢弃. 门控并不改变“沿时间展开再反向传播”的基本训练方式, 它改变的是信息和梯度在时间链路上的流动方式.

问题思考:

- 为什么 RNN 中同一个输入 x_t 可能对应不同输出?
- 梯度截断为什么主要缓解梯度爆炸, 而不能直接解决梯度消失?

6 优化、初始化与正则化

这一节复习 W7 的内容. 这里最重要的是先判断问题属于优化 (optimization) 还是泛化 (generalization). 优化关心训练集上的目标函数能不能被有效降下来; 泛化关心训练好的模型能不能在验证集、测试集和真实新样本上保持稳定表现.

1. 先区分优化和泛化

- 训练损失 (training loss) 是模型在训练集上的平均损失, 验证损失 (validation loss) 是模型在验证集上的平均损失. 若训练损失降不下去, 优先检查学习率、优化器、小批量大小 (batch size)、初始化、归一化和网络结构.
- 若训练损失很低但验证损失高, 才优先考虑泛化控制, 例如正则化、数据增强、模型容量、验证协议、权重衰减 (weight decay)、随机失活 (Dropout) 和早停 (Early Stopping).
- 例如, 若训练损失和验证损失都很高, 直接加入随机失活通常不是第一选择, 因为模型连训练集都没有拟合好. 若训练损失很低而验证损失明显更高, 再考虑权重衰减、随机失活或早停更合理.

2. 训练曲线诊断

- 典型过拟合曲线是训练损失持续下降, 验证损失先下降后上升. 这时应优先保留验证损失最低的轮次作为早停的候选模型, 因为继续训练虽然能降低训练误差, 却可能主要在记忆训练集细节.
- 若训练损失和验证损失基本同步上下震荡, 没有形成“训练变好、验证变差”的分叉, 更像优化不稳定而不是典型过拟合. 常见原因是学习率过大, 参数更新在低损失区域两侧来回跨越, 使训练集和验证集损失一起波动.
- 面对优化不稳定, 首先尝试减小学习率、使用学习率衰减或预热 (warmup)、检查输入归一化和小批量大小; 若出现梯度范数过大, 可使用梯度裁剪限制过大更新. 此时直接加大权重衰减或随机失活往往不是首选, 因为它们主要服务于泛化控制, 可能让尚未稳定的训练更加困难.
- 把随机梯度下降法 (SGD) 换成 Adam 往往会让训练损失下降更快, 但并不自动改善泛化. 如果训练-验证差距继续扩大, 问题仍应从正则化、数据划分、模型容量和早停等角度处理.

3. 常见优化器

- 小批量梯度下降法 (Mini-Batch SGD) 用小批量样本估计梯度, 小批量大小控制梯度噪声和硬件效率. 小批量较小时梯度噪声更大, 但每次更新更便宜; 小批量较大时估计更稳定, 但单次更新成本更高.
- 动量法 (Momentum) 使用历史梯度趋势修正更新方向. 如果普通随机梯度下降法在狭长谷底两侧来回震荡, 动量法可以积累稳定方向上的速度, 减少无效震荡.
- RMSProp 根据历史平方梯度调节不同坐标的有效学习率. 梯度长期较大的坐标会被压小步长, 梯度长期较小的坐标会相对得到更大步长.
- Adam 结合动量法和 RMSProp, 通常是深度学习实验的常用起点, 但不能自动解决过拟合. 优化器主要影响训练过程, 泛化仍需要验证集、正则化和合适模型容量共同控制.

4. 初始化与归一化

- 初始化首先要打破神经元对称性, 其次要控制前向信号和反向梯度的尺度. 如果每层输出方差不断放大或缩小, 深层网络就会更容易出现数值不稳定或梯度消失.
- 打破对称性不只是在避免全零初始化. 如果同一隐藏层中若干神经元的入射权重、偏置和输出连接完全相同, 它们在前向传播中输出相同, 在反向传播中接收相同梯度, 更新后仍然相同, 因而难以分工学习不同特征.
- Xavier 初始化主要适合 Tanh 等近似零中心的激活; He 初始化常用于 ReLU 网络; 正交初始化常用于深层网络或循环隐藏矩阵. 这些方法的共同目标都是让信号在层与层之间传播时尺度不要变化得太剧烈.
- 批量归一化 (Batch Normalization, BN) 在小批量维度上统计均值和方差. 对小批量中第 k 个样本的第 l 层净输入 $\mathbf{z}^{(k,l)}$, 先计算小批量均值 μ_B 和方差 σ_B^2 , 再做

$$\hat{\mathbf{z}}^{(k,l)} = \frac{\mathbf{z}^{(k,l)} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad \mathbf{y}^{(k,l)} = \gamma \odot \hat{\mathbf{z}}^{(k,l)} + \beta.$$

其中 ϵ 用于避免除以 0, γ 和 β 是可学习的尺度与偏移参数. 它们的作用是保留表示能力: 网络可以自己决定是否恢复某些有用的尺度和偏移, 而不是被强行固定为零均值、单位方差.

- 层归一化 (Layer Normalization, LN) 在单个样本内部统计均值和方差. 若某个样本在一层中的隐藏向量为 $\mathbf{z} \in \mathbb{R}^d$, 则

$$\mu_L = \frac{1}{d} \sum_{j=1}^d z_j, \quad \sigma_L^2 = \frac{1}{d} \sum_{j=1}^d (z_j - \mu_L)^2,$$

$$\hat{z}_j = \frac{z_j - \mu_L}{\sqrt{\sigma_L^2 + \epsilon}}, \quad y_j = \gamma_j \hat{z}_j + \beta_j.$$

层归一化也有可学习参数 γ_j, β_j . 它常在序列模型和 Transformer 中按单个样本、单个位置的隐藏维度归一化, 因而不依赖当前测试小批量.

- 需要注意, 批量归一化在训练阶段依赖小批量统计量, 推理阶段通常使用训练过程中累计的均值和方差; 推理时不应继续用测试小批量更新累计统计量, 否则测试样本之间会互相影响, 最终评估也会被污染.

5. 正则化

- 正则化的目标是在拟合训练数据和控制模型复杂度之间折中. 若 $\boldsymbol{\theta}$ 表示全部可学习参数, $\hat{\mathcal{R}}_D(\boldsymbol{\theta})$ 表示训练集 D 上的经验风险, $\Omega(\boldsymbol{\theta})$ 表示对参数复杂度的惩罚函数, $\lambda \geq 0$ 表示正则化强度, 则正则化后的目标可写为

$$\mathcal{J}(\boldsymbol{\theta}) = \hat{\mathcal{R}}_D(\boldsymbol{\theta}) + \lambda \Omega(\boldsymbol{\theta}).$$

这里的 Ω 不是固定某一种函数, 而是“正则项”的占位符. 例如 ℓ_1 正则化常取 $\Omega(\boldsymbol{\theta}) = \|\boldsymbol{\theta}\|_1$, 倾向产生稀疏参数; ℓ_2 正则化常取 $\Omega(\boldsymbol{\theta}) = \frac{1}{2} \|\boldsymbol{\theta}\|_2^2$, 倾向抑制参数过大.

- 在普通随机梯度下降法中, ℓ_2 正则化可解释为权重衰减. 随机失活在训练阶段随机丢弃部分激活, 推理阶段通常关闭, 它迫使网络不要过度依赖某一小部分神经元.
- 早停使用验证集决定何时停止训练. 它的思想是: 当验证集表现开始变差时, 继续降低训练损失可能主要是在拟合训练集细节.

6. 问题: 权重衰减与 ℓ_2 正则化

- 普通随机梯度下降法中, 若直接使用权重衰减:

$$\boldsymbol{\theta}_{t+1} = (1 - \beta) \boldsymbol{\theta}_t - \eta \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}_t).$$

- 若使用 ℓ_2 正则化目标

$$L_{\text{total}}(\boldsymbol{\theta}) = L(\boldsymbol{\theta}) + \frac{\lambda}{2} \|\boldsymbol{\theta}\|^2,$$

则

$$\nabla_{\boldsymbol{\theta}} L_{\text{total}} = \nabla_{\boldsymbol{\theta}} L + \lambda \boldsymbol{\theta}.$$

随机梯度下降更新为

$$\boldsymbol{\theta}_{t+1} = (1 - \eta \lambda) \boldsymbol{\theta}_t - \eta \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}_t).$$

因此在普通随机梯度下降法中令 $\beta = \eta \lambda$, 二者形式等价. 例如 $\eta = 0.1, \lambda = 0.5$ 时, 参数会先乘上 $1 - \eta \lambda = 0.95$, 再减去普通损失的梯度步. 这个等价不要直接推广到 Adam, 因为 Adam 会改变梯度方向和逐坐标尺度.

问题思考:

- 如果训练集损失一直很高, 为什么不应首先把问题归因于过拟合?
- 批量归一化中的 γ 和 β 为什么是必要的可学习参数?

7 注意力与 Transformer

这一节复习 W9 的内容. 注意力机制要解决的问题是: 当一个位置需要形成自己的表示时, 它应该从其他位置取多少信息. QKV 记号把这个过程拆成“提出查询、计算匹配、汇聚内容”三步.

1. 注意力的基本计算

- 给定查询 q 、键 k_i 和值 v_i , 注意力先计算相关性得分, 再经 softmax 得到权重:

$$\alpha_i = \frac{\exp(q^\top k_i / \sqrt{d_k})}{\sum_j \exp(q^\top k_j / \sqrt{d_k})}.$$

输出为值向量的加权和

$$c = \sum_i \alpha_i v_i.$$

- Query 表示要找什么, Key 表示怎样被匹配, Value 表示匹配后取走什么信息. 因此, 注意力权重由 query 和 key 决定, 但最终输出是 value 的加权和.
- 分母中的 $\sqrt{d_k}$ 用于缩放点积. 当维度 d_k 较大时, 未缩放的点积可能幅度很大, softmax 容易过早变得尖锐, 从而影响梯度传播.

2. 自注意力

- 自注意力把同一序列的表示 X 映射为

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V.$$

矩阵形式为

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

- 自注意力让任意两个位置可以直接交互, 但注意力矩阵大小为 $L \times L$, 长序列上计算和显存开销较大.
- 在整段输入已经给定的编码或训练阶段, 自注意力同一层中各位置的 query-key 匹配分数可以同时计算, 因而比普通 RNN 更容易并行. 普通 RNN 的 h_t 依赖 h_{t-1} , 时间步之间存在递推依赖, 不能在每一层中完全同时计算.
- 单纯自注意力本身不包含顺序信息. 如果不加入位置编码, 同一组 token 的顺序被打乱后, 模型很难区分“我喜欢你”和“你喜欢我”这类顺序不同的表达.

3. 例子: 计算第一个位置的 QKV 编码

- 设输入序列长度为 3, 经过线性变换得到

$$Q = [q_1, q_2, q_3], \quad K = [k_1, k_2, k_3], \quad V = [v_1, v_2, v_3].$$

第一个位置的编码只需按三步算:

$$e_{1j} = \frac{\mathbf{q}_1^\top \mathbf{k}_j}{\sqrt{d_k}}, \quad j = 1, 2, 3,$$
$$a_{1j} = \frac{\exp(e_{1j})}{\sum_{\ell=1}^3 \exp(e_{1\ell})}, \quad j = 1, 2, 3,$$
$$\mathbf{b}_1 = \sum_{j=1}^3 a_{1j} \mathbf{v}_j.$$

这类问题的关键是分清: 分数由 query 和 key 决定, 最终编码是 value 的加权和。

4. Transformer

- 一个 Transformer 块通常由多头自注意力、逐位前馈网络、残差连接和层归一化组成。
- 多头注意力的直觉是让不同头关注不同关系。例如一个头可以更关注局部相邻位置, 另一个头可以更关注远距离依赖; 最后再把多个头的信息合并。
- Transformer 仍然需要位置编码, 因为单纯自注意力本身不包含序列顺序。
- 仅编码器结构常用于理解任务, 仅解码器结构常用于自回归生成任务。生成式语言模型的基本过程是读入上下文, 预测下一个 token, 再把该 token 接回上下文。

问题思考:

- 在注意力计算中, 为什么权重由 query 和 key 决定, 但输出要对 value 加权?
- 如果不加入位置编码, 自注意力模型会丢失序列中的哪类信息?

8 大语言模型与工具使用

这一节对应 W8 中的 GPT/LLM 原理与用法讲义。这里不展开完整工程系统, 只保留几个核心判断: 大语言模型本质上怎样生成文字, 为什么会产生幻觉, 以及为什么工具可以帮助它少犯错。

1. 基本原理: 文字接龙

- 大语言模型先把文本切成 token, 再根据已有上下文预测下一个 token 的概率分布。生成回答时, 系统选出一个 token, 把它接回上下文, 再继续预测下一个 token。
- 因此, 一段回答不是一次性从模型内部取出的完整文章, 而是一个 token 一个 token 接出来的。从这个角度看, GPT/LLM 可以先理解为一个规模极大的“文字接龙”模型。
- 预训练让模型学会互联网文本中的语言模式和知识模式; 后训练让模型更像助手一样回答问题。但无论阶段如何变化, 底层仍然在改变“下一个 token 应该是什么”的概率。

2. 为什么会出现幻觉

- 幻觉指模型给出看似流畅、可信, 但事实错误或没有依据的内容。它不是因为模型故意说谎, 而是因为生成目标首先要求输出一个在上下文中像样的 token 序列。
- 当上下文中没有足够事实依据, 或问题涉及模型没有可靠学到的细节时, 模型仍可能沿着最像训练文本的方向继续生成。这会让答案在语言上顺滑, 但在事实上一部分是编出来的。

3. 如何尽量减少幻觉

- 给模型更具体的上下文, 例如原文、数据、代码、题目条件或允许使用的资料范围. 上下文越明确, 模型越不需要凭模糊记忆补全细节.
- 对事实性问题, 要求模型说明依据, 或在不确定时明确说不确定. 对近期、冷门或高风险信息, 应优先使用搜索、文件读取、计算器或代码执行等外部工具核对.
- 对复杂任务, 可以把“生成答案”和“检查答案”分开做: 先让模型给出解法, 再要求它逐条核验条件、单位、来源和中间计算.

4. 模型如何使用工具

- 模型本身只是在上下文中生成 token, 并不能天然访问网页、文件系统或计算器. 工具使用是系统额外提供的能力: 模型先判断需要调用什么工具, 工具返回结果后, 这些结果再进入上下文.
- 工具的作用是把模型的模糊记忆变成可检查的工作记忆. 例如搜索工具提供最新网页, 文件工具提供原始文档, Python 工具提供精确计算, 代码工具提供运行结果.
- 工具不能自动保证答案正确. 关键仍然是模型是否选对工具、是否正确理解工具返回的信息, 以及最后是否把工具结果和问题条件对齐.

问题思考:

- 为什么“下一个 token 预测”既能生成流畅文字, 也会带来幻觉风险?
- 在什么情况下应该让 LLM 使用搜索、文件读取或代码执行工具?

9 小结

- W1-W3 建立机器学习、风险最小化和线性模型.
- W4 说明前馈神经网络、激活函数、反向传播和自动微分.
- W5-W6 分别把神经网络结构扩展到空间数据和序列数据.
- W7 说明训练失败要先区分 optimization 与 generalization, 再选择优化器、初始化、归一化或正则化方法.
- W8 说明 GPT/LLM 的基本原理与使用方式, 包括 token、预训练、后训练、采样、幻觉和工具使用.
- W9 说明注意力用 QKV 机制动态汇聚信息, Transformer 是建立在自注意力之上的深层结构.

问题思考:

- 给定一个新模型, 如何判断它主要改变了模型结构、损失函数还是优化过程?
- 当训练集损失和验证集损失走势不一致时, 应如何判断是优化问题还是泛化问题?
- CNN、RNN 和 Transformer 分别利用了数据中的哪一种结构信息?